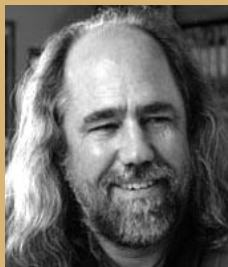


Architectural Organizational Patterns

Grady Booch

A number of elements separate vision from raw, running, naked code; collectively, these are what I've described as the limits of software. For most organizations, the really naughty bits lie at the cusp of design and organization. What is the optimal design for a given system, a design that reasonably balances all the forces that weigh in on



the problem? In turn, what is the optimal organizational structure for developing, deploying, and evolving that system?

The problems of design are largely technical; the problems of organization are largely social. Because the conjunction of these two elements mixes the technical and the social, there's considerable tension at play in all economically interesting projects. Furthermore, that tension is never static. The presence of a running system, even if it's incomplete, uncovers the real risks to the project. It also changes the way its stakeholders see the world, thus impacting the system's design. Similarly, the organization you use to develop an interesting system is rarely the right one for deploying it or even evolving it once it's in place. Injecting a software-intensive system into any context is never a zero-sum event.

To set the context for the discussion that follows, there are some fundamentals worth repeating. I've talked about each of these to some degree in previous columns, so let me summarize here:

- All architecture is design, but not all design is architecture. A system's architecture is defined by its significant design decisions, where in my experience, "significant" is measured by the cost of change.
- Most architectures are accidental; some are intentional.
- Every software-intensive system has an archi-

ture, forged from the hundreds of thousands of small decisions made every day.

- The code is the truth, but not the whole truth. Most architectural information is preserved in tribal memory.
- All well-structured software-intensive systems are full of patterns.

As to that last point, the design patterns literature and practice are both deep as well as practical. Architectural patterns are less well studied, but for our purposes here, think of them as design patterns writ large or as named sets of coherently cooperative design patterns.

Coplien and Harrison's contributions

The observation that all well-structured software-intensive systems are full of patterns speaks to system design, which is a technical problem. We can make a similar observation regarding a development team's social organization, and that's exactly what Jim Coplien and Neil Harrison have done in their brilliant book, *Organizational Patterns of Agile Development* (Prentice Hall, 2004).

This is a tremendously wise book. For example, regarding the intricate dance between architecture and implementation, the authors properly note that "development takes place at two levels: architecture and implementation. Both are ongoing, and they interact with each other strongly. New implementations suggest architectural changes. Architectural changes usually require radical changes to the implementation" (p. 332).

The book focuses on organizational patterns, a number of which are germane to the evolution of a system's architecture. I could not articulate these patterns as well as they have, so I'll let Jim and Neil's work speak for itself.

One of the foundational patterns on which they report is the Developer Controls Process, which observes that projects should "make the developer

the focal point of process information” (p. 71). This makes infinite sense: code is the primary and essential artifact of any meaningful software-intensive system, and code warriors need to be at the center of gravity. Cope and Neil continue by offering a number of architect-related patterns. The Architect Controls Product pattern states that “even though a product is designed by many individuals, a project must strive to give the product breadth to cover its markets and to ensure smooth evolution, but it can’t be blindsided by pragmatic engineering and implementation concerns. Beyond advising and communicating with developers, architects should also participate in implementation” (pp. 254–255). All the world-class architects I’ve had the privilege of working with were also programmers who weren’t afraid of coding.

Recognizing that one architect is singularly insufficient for most complex systems, Cope and Neil posit the Architecture Team pattern: “You need to create an architecture that is simple and cohesive, but that also accommodates a variety of constituencies. Create a small team of ‘resonating minds’ to define the initial architecture in such a way that the team covers the expected partitioning of the system” (pp. 241–242). In my experience, this pattern extends to the problems of systems engineering. In a complex software-intensive system that also involves significant hardware elements (ranging from a spacecraft’s intricacies to globally distributed server farms), such a team should include more than just software stakeholders.

The authors go on to describe several patterns germane to the nature of that architecture team:

- **Lock ’em Up Together:** “A team of diverse people must come up with a single coherent architecture. Gather domain experts together in the same room to work out the architecture (or other strategic issues)” (p. 243).
- **Engage Customers:** “Closely couple the Customer role with the Developer and Architect roles, not just with QA or marketing roles” (p. 113).
- **Stand-Up Meeting:** “Hold short daily meetings with the entire team to exchange critical information, update status, and/or make assignments” (p. 248).

Two other patterns worth mentioning relate to the process of coevolving the system’s architecture and implementation:

- **Named Stable Bases:** “It is important to integrate software frequently enough so that the base doesn’t become stale, but not so frequently that you damage a shared understanding of what functionality is sound and trusted in an evolving software base. Stabilize system interfaces—the architecture—about once a week” (p. 42).
- **Decouple Stages:** “Development stages should be independent to reduce coupling and promote the autonomy of teams and developers” (p. 217).

Finally, Cope and Neil present four patterns relative to the connection between software architecture and social organization:

- **Conway’s Law:** “Make sure the organization is compatible with the product architecture” (p. 192).
- **Organization Follows Location:** “The architectural partitioning should reflect the geographic partitioning, and vice versa” (p. 195).
- **Subsystem by Skill:** “Separate subsystems by staff skills and skill requirements” (p. 153).
- **Feature Assignment:** “Assign features to people for development. Feature development has a finite duration and is, therefore, an assignment, not a role” (p. 265).

Five new patterns

Building on their work, I posit five overarching architectural organizational patterns:

- **Big Dripping Hairball**
- **Senior Designer**
- **Chief Architect**
- **Architecture Team**
- **Architecture Control Board**

Brian Foote and Joseph Yoder identified the Big Ball of Mud pattern as a “haphazardly structured, sprawling, sloppy, duct-tape and bailing wire, spaghetti-code jungle.” Such systems are typically created by developers organized as a Big Dripping Hairball, wherein the resulting system’s architecture is entirely accidental. This isn’t

necessarily an evil pattern; it’s appropriate to prototypes of new systems requiring intense innovation. In such cases, however, we should discard this first prototype, harvest the experience, and apply it to the next version, developed under a more disciplined process.

To build small to modest-sized systems with a modest half-life, projects should identify a Senior Designer to devise an incrementally more intentional architecture drawn from that person’s experience. As the risk to the project—technical and otherwise—grows, projects should identify a Chief Architect; this pattern is appropriate for modest to large-scale systems with a modest to long half-life. Although appropriate to greenfield development, this pattern is especially relevant to brownfield systems requiring extension or transformation.

As complexity and risk continue to grow, an Architecture Team is appropriate for large, complex, high-risk software-intensive systems, particularly where there are many technical, contextual, business, and economic dimensions. In such cases, architectural evolution should be quite intentional. At the largest scales, we need an Architecture Board. This pattern is appropriate for very large systems-of-systems with heavy economic weight, especially systems formed from many systems, some new, many old, and all largely in flux.

As you might observe, these patterns range from low-ceremony, low-intentionality practices to high-ceremony, high-intentionality ones. At the low end, rabid innovation is possible but predictability and repeatability are suspect. At the high end, innovation moves more slowly but predictability and repeatability peak. The low end is liberal and risk embracing, the high end is conservative and risk resistant. The challenge for every organization is to find the sweet spot that provides the right balance of innovation, predictability, repeatability, and risk confrontation for any given project at every given moment. ☞

Grady Booch is an IBM Fellow and one of the UML’s original authors. He also developed the Booch method of software development (*Object-Oriented Analysis and Design*, Addison-Wesley, 1993). He’s working on a handbook of architectural patterns, available at www.booch.com/architecture. Contact him at architecture@booch.com.